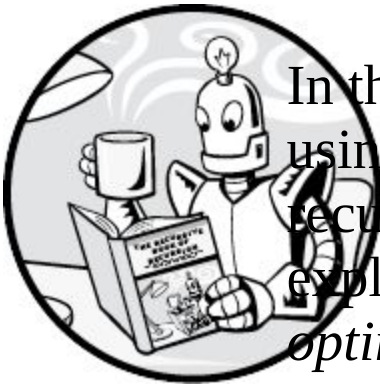


8

TAIL CALL OPTIMIZATION



In the previous chapter, we covered using memoization to optimize recursive functions. This chapter explores a technique called *tail call optimization*, which is a feature provided by a compiler or interpreter to avoid stack overflows. Tail call optimization is also called *tail call elimination*, or *tail recursion elimination*.

This chapter is meant to explain tail call optimization, not to endorse it. I would go so far as to recommend *never* using tail call optimization. As you'll see, rearranging your function's code to use tail call optimization often makes it far less understandable. You should consider tail call optimization to be more of a hack or workaround to make recursion work when you shouldn't be using a recursive algorithm in the first place. Remember, a complex recursive solution is not automatically an elegant solution; simple coding problems should be solved with simple non-recursive methods.

Many implementations of popular programming languages don't even offer tail call optimization as a feature. These include interpreters and compilers for Python, JavaScript, and Java. However, tail call optimization